

粒子玫瑰小白指南

逻辑和浪漫并存，代码与玫瑰共生 —— 用「参数方程 + 粒子系统」复现爆款创意编程作品

学习主题：Creative Coding · 粒子系统 · 参数曲面 | 实验栈：
Python/matplotlib + Three.js | 2026-10-02

这件作品是什么：2026 年 8 月，关智博 (@guanzhiboisme) 在 X 上发布了一条“粒子玫瑰”视频——数万个发光粒子从混沌星云中缓缓汇聚，最终凝成一朵带茎的红玫瑰，获 5.6 万+ 浏览，评论区齐刷刷求教程。作者没放源码，但这类效果是成熟的 **Creative Coding (创意编程)** 主题：**一朵玫瑰 = 一组参数方程算出的“目标点云” + 一场从混沌到目标的粒子旅行**。本指南带你亲手复现它——两套实现（Python 静态版 + Three.js 交互版）全部实测跑通。

疑问清单（本指南的目录）

- 1 粒子玫瑰到底是什么？
- 2 不用建模软件，公式怎么“长出”一朵花？
- 3 一片花瓣的公式是怎么推导的？★核心
- 4 多层花瓣怎么叠才像玫瑰？
- 5 3.4 万个点为什么会“发光”？
- 6 粒子怎么从混沌汇聚成玫瑰？
- 7 为什么动画写在着色器里而不是 JS 里？
- 8 随机数有什么讲究？
- 9 画面为什么会“过曝”？怎么救？
- 10 无头浏览器怎么给 WebGL 拍照验证？
- 11 这套技术和已学的 Three.js 是什么关系？
- 12 学完能拿来做什么？

1

粒子玫瑰到底是什么？

拆开看，“粒子玫瑰”是两个老技术的组合拳：

组成	负责什么	一句话原理
参数曲面 (几何)	回答“粒子最终站哪里”	用数学公式描述玫瑰的表面，在表面上海量采样点
粒子系统 (动画)	回答“粒子怎么动起来”	每个粒子有起点和终点，按时间插值从起点飞到终点

玫瑰负责“美”，粒子负责“动”——把一朵静态数学花变成一场汇聚动画，就是这件作品的全部秘密。没有建模、没有贴图、没有 AI，只有公式和插值。

2

不用建模软件，公式怎么“长出”一朵花？

这正是知识库里已有的**程序化建模**思想：“写菜谱，不捏泥人”。玫瑰的“菜谱”分三段：

花头 四层花瓣（内层小而直立包心 → 外层大而水平外翻），每层用一套花瓣公式

花蕊 中心一小簇深红高斯散点

花茎 一条带正弦微弯的绿色粒子线，从花底垂到 $z \approx -2.7$

类比：**花瓣层像“包烧卖”**——最里面几皮捏得最拢（近乎竖直），越往外皮越摊开，最后几皮几乎平躺在笼屉上。数学上就是让每层的花瓣沿一条“从竖直弯到外翻”的弧线铺开。

3

一片花瓣的公式是怎么推导的？★核心

一片花瓣是一个参数曲面，两个自变量：

$t \in [0,1]$: 沿瓣长走 (0=瓣根贴花心, 1=瓣尖)

$s \in [-1,1]$: 沿瓣宽走 (-1=左缘, 0=中肋, +1=右缘)

分四步搭出来：

① 中心线弧线：瓣根偏离竖直 a_0 、瓣尖偏离 a_1 ，中间线性过渡 $\alpha(t)=a_0+(a_1-a_0)t$ ，投到水平/竖直：

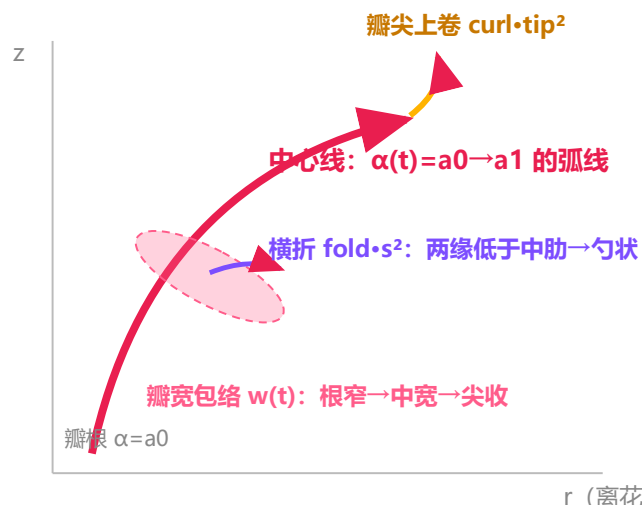
$$r=R \cdot \sin \alpha, z=z_b+R \cdot \cos \alpha$$

② 瓣宽包络：瓣根窄、瓣中宽、瓣尖收：

$w(t)=0.40 \cdot (2\pi/n) \cdot \sin(\pi \cdot t')^{0.7}$ ——乘 $(2\pi/n)$ 让瓣宽随瓣数自适应，瓣多则每瓣窄，留出缝隙不糊团

③ 横向内折： $z += R \cdot \text{fold} \cdot s^2 \cdot t$ ——瓣面两侧比中肋低，呈勺状 (s^2 让中肋处为 0、两缘最大)

④ 瓣尖上卷： $z += R \cdot \text{curl} \cdot \text{tip}^2$ (tip 是 $t>0.55$ 后平滑启动的量)——真实玫瑰外层瓣尖往回勾，这一项是“像不像玫瑰”的点睛之笔



花瓣公式的四个组成部分（侧视示意）

```
# 完整花瓣采样 (Python 版; Three.js 版逐行同款) t = rng.random(n); s = rng.uniform(-1, 1, n) # 随机撒点
alpha = a0 + (a1 - a0) * t # ① 弧线 r = R * np.sin(alpha); zc = zb + R * np.cos(alpha)
w = 0.40*(2\pi/n) * np.sin(\pi*clip(0.12+0.88t))^{0.7} # ② 包络 phi = 瓣朝向 + s * w x, y = r*cos(phi), r*sin(phi)
z = zc + R*fold*s^2*t + R*curl*tip^2 # ③④ 形变 x,y,z += 高斯微抖动(0.012)
# "绒"感
```

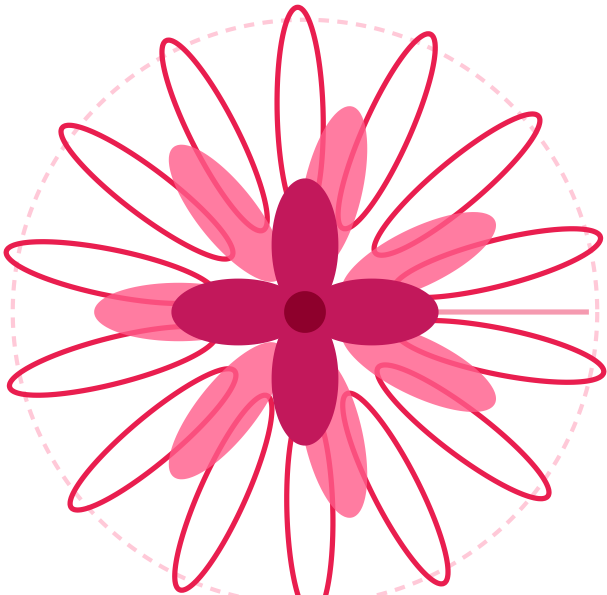
4

多层花瓣怎么叠才像玫瑰？

单层花瓣只是"一个荷叶边"，玫瑰感来自**四层递进** + **三个防糊手段**：

层	瓣数	弧线 $a_0 \rightarrow a_1$	性格
内层	4	$8^\circ \rightarrow 35^\circ$	直立捧心
中内	7	$22^\circ \rightarrow 56^\circ$	半拢
中外	10	$40^\circ \rightarrow 76^\circ$	摊开
外层	14	$58^\circ \rightarrow 93^\circ$	近水平外翻+尖卷

防糊三招：① 瓣宽 $\propto (2\pi/\text{瓣数})$ ，瓣多的层瓣窄，留缝；② 每层相位 phase 错开 ($\pi/7$ 、 $\pi/4$ 、 $\pi/16$)，上下层花瓣互相插空；③ 内层 z_{base} 抬高，让花心"凸"出来。



俯视：三层相位错开、外层瓣多而窄

俯视分层示意（示意环，非真实渲染）

"包心结构"是玫瑰和荷花、蒲公英在公式层面的本质区别：瓣尖角 a_1 从内向外逐层增大 ($35^\circ \rightarrow 93^\circ$)，一朵花里藏着一条"从抱拢到放开"的连续谱。

5 3.4 万个点为什么会"发光"？

Three.js 里所有 3.4 万个点装进一个 `THREE.Points` 对象（场景图里的一个节点），发光感来自三个渲染开关的组合：

技术	作用	不开会怎样
点精灵 <code>gl_PointSize</code>	每个顶点渲染成一个带纹理的正方形面片（不是 1 像素点）	粒子小成沙砾，没有形体感
软边圆形 <code>smoothstep</code>	片元着色器里按到点中心距离做圆形淡出	每个粒子是硬边方块，放大有"马赛克"感
加法混合 <code>AdditiveBlending</code>	重叠粒子的颜色相加——叠得越多越亮，向着白色逼近	画面像平平的纸片，没有光感

加法混合 = "光的叠照"：两束光重叠处更亮。密集处自动过亮成白、稀疏处透出深红——粒子云的"辉光层次"是免费的。

配套纪律：开加法混合必须配 `depthWrite:false`（粒子不写深度），否则先画粒子会挡住后面的粒子，透明就失效了。

对比：matplotlib 版没有这些（普通散射点），所以同一样公式，matplotlib 版像"绒布剪影"，Three.js 版像"霓虹雕塑"——**数学相同，观感差距全在渲染开关上。**

6 粒子怎么从混沌汇聚成玫瑰？

每个粒子带两套坐标，存在同一条几何体的两个属性里：

aStart：混沌起点——半径 6 左右的大球壳随机点（略向下偏，像一片星云）

aTarget：玫瑰目标点——Q3/Q4 的公式算出的位置

动画每帧只更新一个数：**全局进度 uProgress**：
0→1。顶点着色器里：

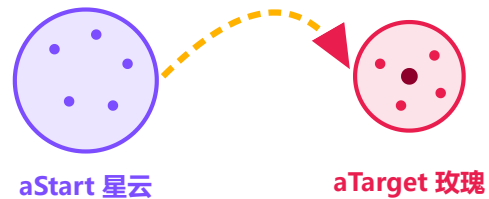
```
float p = clamp((uProgress - delay) / 0.65, 0., 1.); p = easeInOutCubic(p); // 缓动 vec3 pos = mix(aStart, aTarget, p);
```

两个灵魂细节：

- ① **每粒子随机延迟 delay (0~0.35)** ——粒子不是齐步走，像潮水般此起彼伏地抵达；
- ② **缓动函数 easeInOutCubic**——起步慢、中段快、到位前减速“刹车”，机械感秒变生命感。

到位后再叠加 **sin(时间+种子)** 的微小呼吸漂移——花就“活”了。

mix(start,target,ease(p))



每个粒子：星云 → (缓动插值) → 玫瑰上的专属座位

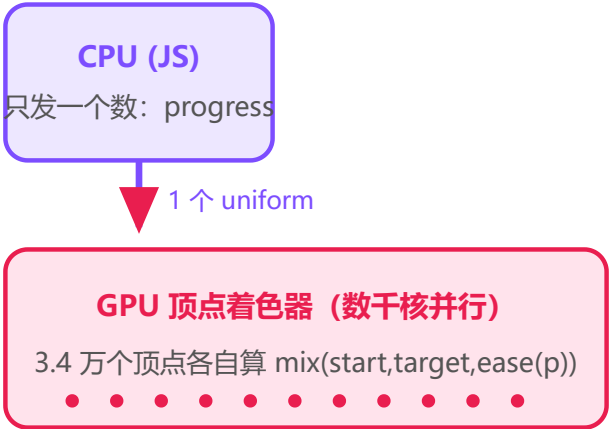
7

为什么动画写在着色器里而不是 JS 里？

3.4 万粒子 × 每帧 60 次刷新 = 每秒 200 万次位置更新。两个做法的天壤之别：

	CPU (JS 逐帧改坐标)	GPU (顶点着色器)
数据在哪	内存 ↔ 显存来回搬	常住显存，零搬运
谁算	JS 主线程串行	数千核心并行
每帧成本	3.4万次循环+上传	改 1 个 uniform 变量

着色器方案：JS 每帧只更新 uProgress 一个数（从 0 走到 1），3.4 万粒子的位置全部由 GPU 各自算——CPU 几乎零负担，还稳 60 帧。



CPU 发令、GPU 齐算：并行粒子动画的标准架构

8 随机数有什么讲究？

两个容易忽略但决定成败的细节：

① **固定种子 = 可复现**。JS 里 `Math.random()` 没法设种子，每次刷新是另一朵花；实验用 `mulberry32(42)` 自实现可播种随机数——**种子固定，粒子云 100% 复现，改一个参数前后对比才有意义**（这也呼应知识库的“确定性脚本”概念：同输入同输出，才好当实验品）。

② **高斯抖动 ≠ 均匀抖动**。花瓣表面叠加 `gauss()×0.012` 的微小偏移（Box-Muller 变换生成正态分布）：均匀抖动像“沙纸”，高斯抖动中心密、边缘稀，才有**绒雾般的体积感**。

坑：把抖动调到 0.05 以上花瓣轮廓就散架了；“绒感”要的是 1% 量级的扰动，不是噪声。

9 画面为什么会“过曝”？怎么救？

实测第一版 Three.js 截图整个花头糊成一张白饼。原因和药方：

病因	药方（本次实测有效）
点尺寸 <code>uSize=3.2</code> 太大，相邻粒子面片大量重叠，加法叠加爆白	降到 0.62 （经验：加法混合下点宜小而多，不宜大而糊）
每个粒子不透明度太高	软边 alpha 乘 0.55
中心亮核 + 辉光增益过猛	亮核 0.35→ 0.10 ，颜色增益 (0.75+0.6g)→(0.80+0.5g)

加法混合的调参心法：亮度是“叠”出来的，单粒子要刻意调暗；单看每个粒子灰暗得可惜，合在一起才刚刚好。

粒子玫瑰 • Three.js

鼠标拖动旋转 · 滚轮缩放

粒子数 34,400 · 进度 100%



调参后：层次分明的发光玫瑰

10

无头浏览器怎么给 WebGL 拍照验证？

实验程序"是否真的画出了玫瑰"不能靠嘴说——用 Edge 无头模式自动截图当证据（呼应知识库"证据优先质检"：看实机画面，不看日志）：

```
msedge --headless=new --disable-gpu --use-angle=swiftshader \ --window-size=900,1000 --
virtual-time-budget=9000 \ --screenshot="rose_final.png" \ "file:///F:/reminder/particle-
rose/exercise/rose.html?final=1"
```

三个关键开关：

- ① `--use-angle=swiftshader`：无 GPU 环境用 CPU 软渲染 WebGL（无头模式下 `--disable-gpu` 会禁真硬件加速，SwiftShader 兜底）；
- ② `--virtual-time-budget=9000`：把页面的时钟快进 9 秒——rAF 动画瞬间跑完 5.2 秒的汇聚，不用真等；
- ③ `?final=1`：页面内置的跳关参数，直接把进度钉在 100%，截图结果确定。

✅ **实测：**该命令在本机真实输出 rose_final.png（87KB→调参后 221KB），画面为完整发光玫瑰；把 budget 改成 3200 得到 21% 进度的混沌星云图——**一条命令即可抽取动画任意时刻的确定性帧。**

11 这套技术和已学的 Three.js 是什么关系？

对照知识库已有概念：

已有概念	本项目的新增量
[[Three.js与场景图]]	Points 只是场景图里又一个节点；但动画发生在 顶点着色器 里，完全绕过场景图的变换层级——场景图管"静态组织"，着色器管"海量并行"
[[程序化建模]]	上一课用代码拼"积木"（几何体+布尔），本课用参数方程描述 连续曲面 ——菜谱更细了，细到每个点的坐标都有公式
[[确定性脚本]]	固定随机种子让粒子云可复现，是参数实验的对照前提
[[证据优先质检 ProofOverClaims]]	无头截图 = AI/自动化流程的"眼睛"，WebGL 页面同样适用

新概念两条：**粒子系统**（起点/终点双缓冲+缓动插值的群体动画范式）与**参数曲面采样**（两个参数的方程描述三维表面）。

12

学完能拿来做什么？

玩法	怎么改
节日表白页 / 贺卡	把 aTarget 换成心形参数方程（已有大量开源公式），其余一行不改
粒子烟花	目标点改"爆炸球面"，缓动改 ease-out，加重力项
星系/星云可视化	目标点按对数螺旋线分布（旋臂），颜色按半径渐变
Logo / 文字粒子动画	aTarget 从 Canvas 2D 读像素点坐标——任何图形都能"粒子化"
数据雕塑	目标点=你的数据（如词频、传感器序列），把抽象数据变成可旋转的形体

一套"起点+目标+缓动"的管线，换掉目标点云，就是另一个作品——这就是 Creative Coding 的杠杆率。



动手实验：亲手种出一朵粒子玫瑰（已实测通过）

实验目标：从零跑通"数学验证 → 交互动画 → 参数改造 → 自动化取证"四步，实验材料在 `particle-rose\exercise\`。

1

准备：拿到 three.js 并放进本地 vendor 目录

操作：

```
curl -sL -o exercise/vendor/three.min.js  
https://unpkg.com/three@0.128.0/build/three.min.js
```

目的：本地化依赖，双击 HTML 就能跑（不走 `npm install`，也不联网）。

为什么是 r128：这是带 UMD 版（`three.min.js` 全局变量）的最后几个版本之一——**新版只有 ES Module**，而 ES Module 在 `file://` 协议下会被 CORS 拦截，双击打不开。这是新手最常见的"明明没错却黑屏"的坑。

✓ **验证：** `wc -c three.min.js` 输出 603445 字节，文件头含 "three.js Authors"。

2

先用 Python 验证玫瑰公式（数学先于动画）

操作：

```
pip install matplotlib numpy → python exercise/rose_mpl.py
```

目的：动画会掩盖公式错误，先用最朴素的 3D 散点图确认"玫瑰长得对不对"。

实现：四层弧线花瓣（4/7/10/14 瓣）+ 花蕊 + 花茎，共 34,400 粒子，一次性渲染成静态图。

✓ **预期结果：**输出 `rose_mpl.png`（实测 34400 粒子，约 3 秒出图）：黑底上一朵层叠红玫瑰、绿色弯茎。若改 `a1` 各层角度，花型立刻变（比如全改 90° 变成平盘子——这就是"包心结构"的对照实验）。

3

打开 Three.js 交互版，看混沌汇聚动画

操作：双击 `exercise/rose.html`（或命令行 `start exercise\rose.html`）。

目的：体验完整作品：34,400 粒子从星云汇聚成玫瑰，5.2 秒，鼠标可拖动旋转、滚轮缩放。

实现：同一套公式 + GPU 着色器动画（Q6/Q7）。

✅ **预期结果：**左上角 HUD 显示“粒子数 34,400 · 进度 xx%”；进度走到 100% 后呈现发光玫瑰，与下图图中图一致。

4

改参数，看公式如何回应（成就感时刻）

操作 & 预期：在浏览器地址栏的 URL 后面加参数刷新：

URL	现象（均已实测截图）
<code>rose.html?k=0.55</code>	每层瓣数减半（4/7/10/14 → 2/4/6/8），花瓣变大变肥， rose_k06.png
<code>rose.html?blue=1</code>	红玫瑰变蓝玫瑰（颜色通道整体映射），rose_blue.png
源码里改 <code>DURATION = 5200</code>	汇聚变快/变慢，动画节奏随你掌控

✅ **验证标准：**每次改参数刷新，花型/颜色应立刻变化且**每次刷新形状相同**（固定种子的可复现性，Q8）。

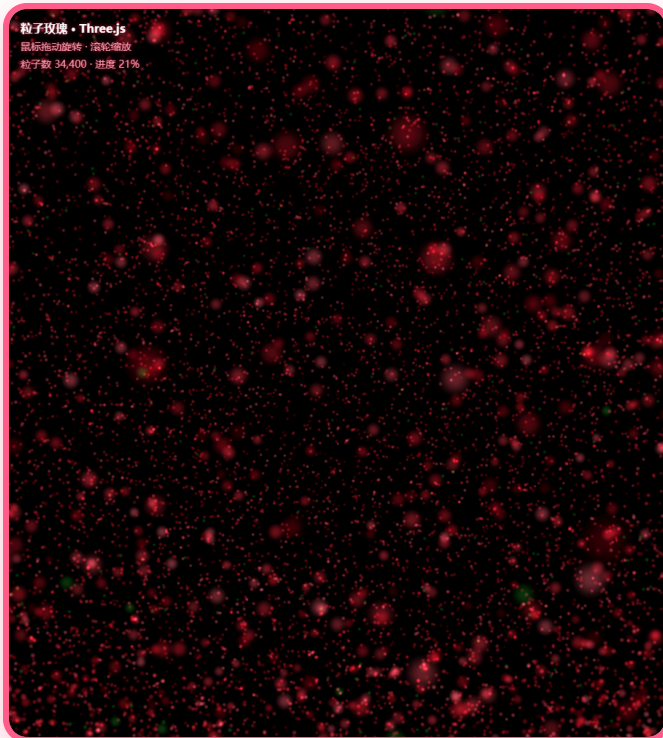
5

无头截图：给作品拍“确定性”的证件照

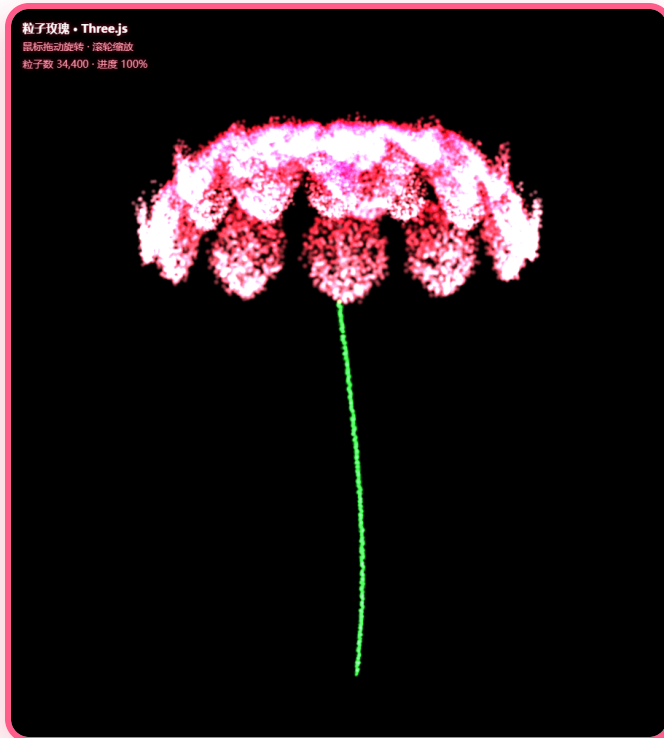
操作：运行 Q10 的 msedge 无头命令。

目的：把“我画出来了”变成可存档、可对比的 PNG 证据——这是所有 AI/自动化图形工作流的验收基建。

✅ **预期结果：**rose_final.png（100% 成品）、rose_morph.png（21% 混沌星云）两张确定性帧落盘，尺寸 900×1000。



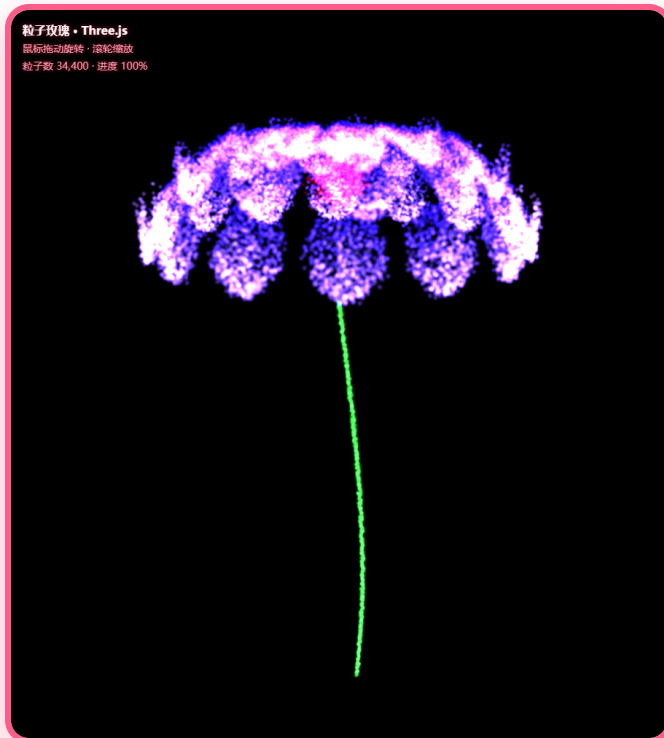
① 21%: 混沌星云 (起点态)



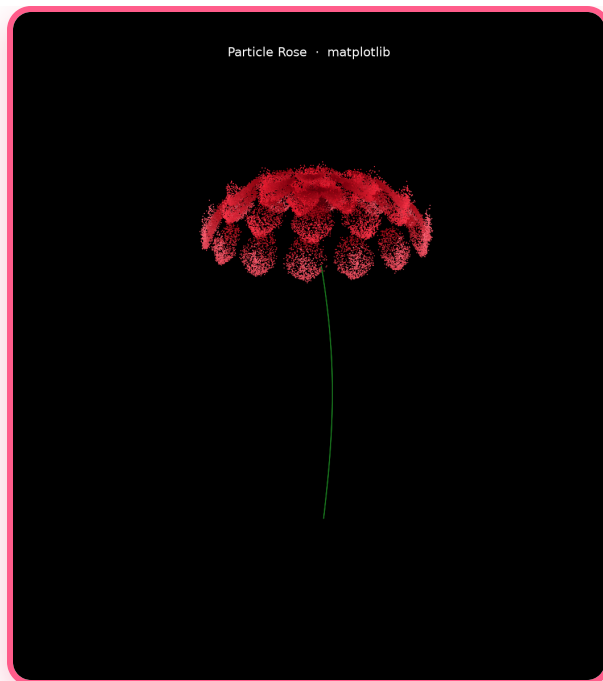
② 100%: 汇聚完成的发光玫瑰



③ $k=0.55$: 瓣数减半的实验组



④ $blue=1$: 蓝色变体



⑤ Python/matplotlib 版：同一公式的朴素验证

一句话带走：粒子玫瑰 = 参数方程给出"美的坐标" + 缓动插值负责"浪漫的抵达"。公式管形，时间管情，GPU 管量——逻辑和浪漫并存，这句话在代码层面是字面成立的。